

The Chromatic Lands: Comprehensive Game Design Document

1. Game Overview

Core Concept

The Chromatic Lands is a free-to-play educational RPG where players learn music theory through an epic adventure. Players assist Piano and Forte from another dimension by learning musical patterns that become magical spells. The game adapts to each player's learning pace using evidence-based educational algorithms.

Mission Statement

Provide comprehensive music theory education to all students regardless of financial situation, using Gordon's Music Learning Theory principles in an engaging, story-driven format.

Target Audience

- Primary: Students ages 8-14
- Secondary: Music educators seeking supplemental teaching tools
- Tertiary: Parents wanting quality educational content

Platform

- HTML5 web-based (Construct 3)
- Optimized for Chromebooks
- Playable on any modern browser

2. Core Gameplay Loop

Episode Structure

- **Length:** 5-10 minutes per episode
- **Patterns per Episode:** 3-5 new patterns introduced
- **Season Length:** 5-7 episodes bundled together
- **Flow:** Story → Practice → Resonance Gate → Story Continuation

Daily Practice Flow

1. **Check Resonance Gate** - Shows if player is ready for story continuation
2. **Practice Available Patterns** - Through mini-games and exercises

3. **Progress Story** - When gate requirements are met
4. **Complete Episode Challenge** - Boss battle or action sequence

Resonance Gate System

Magical barriers that test musical readiness before dangerous areas:

- **Opens When:**
 - 85% confidence on required patterns
 - No patterns below 70% confidence
 - Completed minimum spaced repetition reviews
- **Visual Feedback:**
 - Gate glows based on readiness level
 - Musical notes float when ready
 - Different designs for rhythm/tonal gates
- **Dimensional Power Explanation:** "The dimensional rift is weakening, you must return to recharge your connection" (prevents overplay if needed)

3. Character System

Main Characters

- **Piano:**
 - Class: Support/healer
 - Personality: Small, quiet (reinforces "piano" dynamic)
 - Weapon: Tiny lyre
 - Magic focus: Healing and support spells
- **Forte:**
 - Class: Attacker
 - Personality: Loud, bold (reinforces "forte" dynamic)
 - Weapon: Large guitar
 - Magic focus: Damage and offensive spells
- **Player:**
 - Named by user at start
 - Assists from our dimension through computer portal
 - Never appears on screen

- Addressed by name in text, pronouns in voiced cutscenes

Mascot

- **Echoling:**
 - Forte's pet
 - Tiny dragon that helps with pattern practice
 - Echoes patterns back to player
 - Provides hints when players struggle
 - Appears when player needs help or encouragement

Character Progression

- Experience points from successful pattern completion
- Abilities unlock based on mastered intervals
- Stats modifiers improve with practice consistency
- Level progression tied to pattern mastery

Con Man Wizard

- Mysterious character who "accidentally" helps
- Appears when players struggle (3-5 failed attempts)
- Provides scaffolding without breaking immersion
- Explains concepts through "mishaps" and "accidents"

4. Magic System

Interval-Based Spells

All 12 interval types create different magical effects:

- **Unison:** Basic attack, neutral damage
- **Minor 2nd:** Dissonance damage, confusion effects
- **Major 2nd:** Movement/speed buffs
- **Minor 3rd:** Sadness effects, enemy debuffs
- **Major 3rd:** Joy magic (summons illusions like puppies, healing)
- **Perfect 4th:** Stability shields, defense boost
- **Tritone:** Chaos magic, unpredictable effects

- **Perfect 5th:** Strong defensive barriers, harmony shields
- **Minor 6th:** Melancholy effects, time slow
- **Major 6th:** Warmth/comfort healing over time
- **Minor 7th:** Tension building, charge attacks
- **Major 7th:** Resolution magic, powerful finishers

Spell Casting Mechanics

- Select interval on instrument display
- Input via spacebar, controller, or MIDI keyboard
- Accuracy affects spell power:
 - 95-100%: Full power + bonus effects
 - 85-94%: Full power
 - 70-84%: Reduced power
 - Below 70%: Spell may fail or backfire
- Response time affects casting speed:
 - Under 2 seconds: Fast cast
 - 2-4 seconds: Normal cast
 - Over 4 seconds: Slow cast
- Wrong intervals create specific effects based on what was played

5. Adaptive Learning System

Core Algorithm

Confidence Score Calculation

$\text{Confidence} = \text{Base Accuracy} \times \text{Speed Bonus} \times \text{Consistency Bonus}$

Where:

- Base Accuracy = $\text{correct_attempts} / \text{total_attempts}$

- Speed Bonus:

- < 2 seconds: 1.2x

- 2-4 seconds: 1.0x

- > 4 seconds: 0.8x

- Consistency Bonus:

- 1 correct in row: 0.6x

- 3 correct in row: 1.0x

- 5+ correct in row: 1.2x

Pattern Progression States

1. **Unknown:** Pattern shows as ????? (dark/locked)
2. **Aural Recognition:** Shows pattern as "Bum" (rhythm) or "Bah" (tonal)
3. **Verbal Association:** Shows with proper syllables (Du-de, Do-Re-Mi)

Mastery Criteria

- **Struggling:** Confidence below 0.70
- **Learning:** Confidence 0.70-0.84
- **Mastered:** Confidence 0.85+ with 3+ streak

Spaced Repetition System

- **Initial Learning:** Must reach 95% accuracy
- **Review Schedule:**
 - 1st review: 2 days
 - 2nd review: 4 days
 - 3rd review: 7 days
 - 4th review: 14 days
 - 5th+ review: 30 days

Session Design (70/30 Rule)

- **70% Review:** Previously learned patterns
- **30% New:** Maximum 2 new patterns per session

- **Cognitive Load:** 3-5 patterns total per session
- **Optimal Duration:** 10-15 minutes

Difficulty Adjustment

- **Monitor:** Last 10 attempts in category
- **Increase Difficulty:** Average accuracy > 95% for 10+ attempts
- **Decrease Difficulty:** Average accuracy < 70% for 10+ attempts
- **Maintain:** Average accuracy between 70-95%

6. Implementation Functions (Construct 3)

Function: UpdatePatternProgress

INPUT: patternId, wasCorrect, responseTime

1. IF pattern doesn't exist in save file:

- Add pattern with default values
- Set category based on patternId prefix (rhythm_ or tonal_)
- Set difficulty based on Gordon's taxonomy

2. Get current pattern data

3. Update attempt count:

- Add 1 to attempts

4. Calculate accuracy:

- IF wasCorrect:
 - Add 1 to correct attempts
- New accuracy = correct attempts / total attempts

5. Update streak:

- IF wasCorrect:
 - Add 1 to current streak
- ELSE:
 - Reset streak to 0

6. Calculate speed bonus:

- IF responseTime < 2 seconds: speedBonus = 1.2
- ELSE IF responseTime <= 4 seconds: speedBonus = 1.0
- ELSE: speedBonus = 0.8

7. Calculate consistency bonus:

- IF streak = 1: consistencyBonus = 0.6
- ELSE IF streak = 3: consistencyBonus = 1.0
- ELSE IF streak >= 5: consistencyBonus = 1.2
- ELSE: interpolate between values

8. Calculate confidence:

- confidence = accuracy × speedBonus × consistencyBonus

9. Update pattern data with new values

10. Schedule next review:

- IF confidence >= 0.85 AND streak >= 3:
 - Pattern is mastered
- Schedule review based on review count:
 - 1st review: 2 days
 - 2nd review: 4 days

- 3rd review: 7 days
- 4th review: 14 days
- 5th+ review: 30 days
- ELSE:
- Keep in active practice pool

Function: GetNextPatterns

INPUT: skillCategory (rhythm or tonal), sessionLength

1. Get all unlocked patterns in category
2. Separate patterns into buckets:
 - Due for review (past nextReview date)
 - Struggling (confidence < 0.70)
 - Learning (confidence 0.70-0.84)
 - New (never attempted)
 - Mastered (confidence 0.85+, not due for review)
3. Calculate pattern distribution:
 - Total patterns = sessionLength / 30 seconds per pattern
 - Review patterns = Total × 0.70
 - New patterns = Total × 0.30 (max 2)
4. Build pattern list:
 - FIRST: Add all due for review patterns
 - SECOND: Add struggling patterns
 - THIRD: Add learning patterns
 - FOURTH: Add new patterns (up to limit)
 - FIFTH: If still need patterns, add random mastered
5. Shuffle pattern order to mix difficulty
6. Return pattern list

Function: CheckDifficultyAdjustment

INPUT: skillCategory

1. Get last 10 attempts in category
2. Calculate recent performance:
 - Average accuracy
 - Average response time
 - Success rate
3. Check if adjustment needed:
 - IF average accuracy > 0.95 for 10+ attempts:
 - Increase difficulty level
 - ELSE IF average accuracy < 0.70 for 10+ attempts:
 - Decrease difficulty level
 - Add scaffolding hints
4. IF difficulty changed:
 - Update adaptiveProfile
 - Log adjustment event
 - Select patterns from new difficulty

Function: HandleStrugglingPattern

INPUT: patternId, attemptHistory

1. Check struggle severity:

- Count consecutive failed attempts
- Check if confidence stuck below 0.50

2. IF failed 3-5 times:

- Trigger "Con Man Wizard" hint
- Wizard "accidentally" explains concept
- Add to reteachingMoments

3. IF failed 6+ times:

- Unlock special mini-game for that pattern
- Simpler context, more scaffolding
- Add to unlockedMiniGames

4. IF pattern involves intervals:

- Check commonErrors data
- Create targeted practice:
 - "Perfect 4th vs Perfect 5th Challenge!"
 - Focus on distinguishing confused intervals

5. Log intervention in struggleInterventions

Function: GenerateMaestroReport

INPUT: playerData

1. Check each skill category performance:
 - IF rhythm averageAccuracy > 0.90:
 - Add encouragement: "Your timing is rock solid!"
 - IF tonal averageAccuracy > 0.85:
 - Add encouragement: "Your pitch recognition is improving!"
2. Identify struggling patterns:
 - FOR each pattern with confidence < 0.70:
 - Add to strugglingFocus with friendly message
 - Example: "The 'so-mi' interval is tricky - let's practice!"
3. Check recent achievements:
 - IF new pattern mastered (confidence >= 0.85):
 - Add achievement celebration
 - IF streak milestone reached (5, 10, 20):
 - Add streak recognition
4. Return formatted report with:
 - 2-3 encouragements
 - 1-2 focus areas (if any)
 - Recent achievements

Function: RecordIntervalError

INPUT: correctInterval, playerAnswer

1. Get interval data from adaptiveProfile
2. Update commonErrors:
 - IF playerAnswer exists in commonErrors:
 - Add 1 to count
 - ELSE:
 - Create new entry with count = 1
3. IF any error occurs 3+ times:
 - Flag for targeted practice
 - Create comparison exercise
4. Generate insight:
 - "You often confuse Perfect 4th with Perfect 5th"
 - Suggest specific practice game

Function: GenerateDailyPractice

1. Check which categories player has unlocked
2. FOR each unlocked category:
 - Get patterns due for review today
 - Get struggling patterns (confidence < 0.70)
3. Create mini-sessions:
 - 5 minutes rhythm patterns
 - 5 minutes tonal patterns
 - 5 minutes mixed review
4. Apply 70/30 rule to each mini-session
5. Return structured practice plan

Function: SaveSessionResults

INPUT: sessionData

1. Create session record with:
 - Unique sessionId
 - Current timestamp
 - Session type (episode, practice, assessment)
 - Duration
2. FOR each activity in session:
 - Record pattern attempts
 - Record vocabulary attempts
 - Calculate session accuracy
3. Check for adaptive events:
 - Difficulty adjustments
 - New patterns mastered
 - Struggling patterns identified
4. Add to sessionHistory (keep last 100)
5. Update global stats:
 - Total play time
 - Session count
 - Last played date

7. Save File Structure


```
{
  "version": "1.0.0",
  "playerId": "unique_player_id",
  "playerName": "Student Name",
  "lastModified": "timestamp",

  "globalStats": {
    "totalPlayTime": 0,
    "totalSessions": 0,
    "lastPlayed": "timestamp",
    "currentArc": 1,
    "currentEpisode": "1-1",
    "unlockedArcs": [1],
    "instrumentDisplay": 0
  },

  "learningData": {
    "unlockedCategories": ["rhythm", "tonal_major", "vocabulary", "techniques"],

    "patterns": {
      "rhythm_macro_microbeat_duple": {
        "patternId": "rhythm_macro_microbeat_duple",
        "category": "rhythm",
        "difficulty": "easy",
        "stage": {
          "neutral": {
            "confidence": 0.85,
            "attempts": 5,
            "streak": 3
          },
          "syllables": {
            "confidence": 0.0,
            "attempts": 0,
            "streak": 0
          }
        }
      },
      "lastSeen": "2024-01-01T00:00:00Z",
      "nextReview": "2024-01-03T00:00:00Z",
      "reviewCount": 1
    },
    "tonal_major_tonic_do_mi_so": {
      "patternId": "tonal_major_tonic_do_mi_so",
      "category": "tonal_major",
```

```
"difficulty": "easy",
"stage": {
  "neutral": {
    "confidence": 0.72,
    "attempts": 3,
    "streak": 2
  },
  "syllables": {
    "confidence": 0.0,
    "attempts": 0,
    "streak": 0
  }
},
"lastSeen": "2024-01-01T00:00:00Z",
"nextReview": "2024-01-03T00:00:00Z",
"reviewCount": 0
},
```

```
"vocabulary": {
  "forte": {
    "termId": "forte",
    "category": "dynamics",
    "definition": {
      "understood": true,
      "attempts": 2
    },
  },
  "audioRecognition": {
    "accuracy": 0.9,
    "attempts": 5
  },
  "lastSeen": "2024-01-01T00:00:00Z",
  "nextReview": "2024-01-08T00:00:00Z"
},
"staccato": {
  "termId": "staccato",
  "category": "articulation",
  "definition": {
    "understood": true,
    "attempts": 1
  },
  "audioRecognition": {
    "accuracy": 0.85,
    "attempts": 3
  }
}
```

```
},  
  "lastSeen": "2024-01-01T00:00:00Z",  
  "nextReview": "2024-01-08T00:00:00Z"  
}  
},
```

```
"adaptiveProfile": {  
  "rhythm": {  
    "currentLevel": "easy",  
    "averageAccuracy": 0.82,  
    "averageSpeed": 2.3,  
    "strugglingPatterns": [],  
    "masteredPatterns": ["rhythm_macro_microbeat_duple"],  
    "suggestedDifficulty": "easy"  
  },
```

```
  "tonal": {  
    "currentLevel": "easy",  
    "averageAccuracy": 0.75,  
    "averageSpeed": 3.1,  
    "strugglingPatterns": ["tonal_major_tonic_do_mi_so"],  
    "masteredPatterns": [],  
    "suggestedDifficulty": "easy"  
  },
```

```
  "intervals": {  
    "currentLevel": "beginner",  
    "byInterval": {  
      "unison": {  
        "accuracy": 0.0,  
        "attempts": 0,  
        "avgResponseTime": 0,  
        "commonErrors": {}  
      },
```

```
      "minor2nd": {  
        "accuracy": 0.0,  
        "attempts": 0,  
        "avgResponseTime": 0,  
        "commonErrors": {}  
      },
```

```
      "major2nd": {  
        "accuracy": 0.0,  
        "attempts": 0,  
        "avgResponseTime": 0,  
        "commonErrors": {}  
      },
```



```
"minor3rd": {
  "accuracy": 0.0,
  "attempts": 0,
  "avgResponseTime": 0,
  "commonErrors": {}
},
"major3rd": {
  "accuracy": 0.75,
  "attempts": 4,
  "avgResponseTime": 3.2,
  "commonErrors": {
    "perfect4th": 1
  }
},
"perfect4th": {
  "accuracy": 0.0,
  "attempts": 0,
  "avgResponseTime": 0,
  "commonErrors": {}
},
"tritone": {
  "accuracy": 0.0,
  "attempts": 0,
  "avgResponseTime": 0,
  "commonErrors": {}
},
"perfect5th": {
  "accuracy": 0.0,
  "attempts": 0,
  "avgResponseTime": 0,
  "commonErrors": {}
},
"minor6th": {
  "accuracy": 0.0,
  "attempts": 0,
  "avgResponseTime": 0,
  "commonErrors": {}
},
"major6th": {
  "accuracy": 0.0,
  "attempts": 0,
  "avgResponseTime": 0,
  "commonErrors": {}
},
```

```
    "minor7th": {
      "accuracy": 0.0,
      "attempts": 0,
      "avgResponseTime": 0,
      "commonErrors": {}
    },
    "major7th": {
      "accuracy": 0.0,
      "attempts": 0,
      "avgResponseTime": 0,
      "commonErrors": {}
    }
  }
}
```

```
"playerFeedback": {
  "maestroReport": {
    "lastGenerated": "2024-01-01T00:00:00Z",
    "encouragements": [],
    "strugglingFocus": [],
    "achievements": []
  },
  "struggleInterventions": {
    "triggeredTutorials": [],
    "unlockedMiniGames": [],
    "reteachingMoments": []
  }
},
```

```
"sessionHistory": [
  {
    "sessionId": "session_001",
    "date": "2024-01-01T00:00:00Z",
    "type": "episode",
    "episodeId": "1-1",
    "duration": 300,
    "activities": [
      {
        "activityType": "pattern_practice",
        "patterns": [
          {
            "patternId": "rhythm_macro_microbeat_duple",
```

```
    "accuracy": 0.85,
    "responseTime": 2.1,
    "correct": true
  }
],
},
{
  "activityType": "vocabulary_quiz",
  "terms": [
    {
      "termId": "forte",
      "questionType": "definition",
      "correct": true,
      "responseTime": 3.5
    }
  ]
},
],
"adaptiveEvents": [
  {
    "type": "difficulty_adjusted",
    "reason": "high_accuracy",
    "from": "easy",
    "to": "moderate"
  }
]
},
],

"spacedRepetition": {
  "algorithm": "modified_leitner",
  "boxes": {
    "box1": ["rhythm_macro_microbeat_duple"],
    "box2": [],
    "box3": [],
    "box4": [],
    "box5": []
  },
  "reviewSchedule": {
    "2024-01-03": ["rhythm_macro_microbeat_duple", "tonal_major_tonic_do_mi_so"],
    "2024-01-08": ["forte", "staccato"]
  }
},
],
```

```
"adaptiveSettings": {
  "difficultyAdjustment": {
    "enabled": true,
    "targetAccuracy": 0.85,
    "adjustmentThreshold": 0.15,
    "minSessionsBeforeAdjust": 3
  },
  "spacedRepetitionIntervals": {
    "pattern": [2, 4, 7, 14, 30],
    "vocabulary": [3, 7, 14, 30, 90]
  },
  "sessionBalance": {
    "reviewPercentage": 0.70,
    "newContentPercentage": 0.30,
    "maxNewPatternsPerSession": 2
  }
},

"characterProgress": {
  "piano": {
    "level": 1,
    "experience": 0,
    "unlockedAbilities": ["heal_minor2nd"],
    "statsModifiers": {
      "tonalBonus": 1.0,
      "rhythmBonus": 1.0
    }
  },
  "forte": {
    "level": 1,
    "experience": 0,
    "unlockedAbilities": ["attack_major3rd"],
    "statsModifiers": {
      "tonalBonus": 1.0,
      "rhythmBonus": 1.0
    }
  }
},

"settings": {
  "scaffoldingLevel": 2,
  "musicVolume": 0.8,
  "sfxVolume": 0.8,
  "autoRecord": true,
```

```
"peerReview": false
```

```
}
```

```
}
```

8. Content Progression

Skill Constellation System

Patterns organized in interconnected constellations rather than linear trees:

RHYTHM CONSTELLATION

Duple Patterns ★—★—★—★



Triple Patterns ☆—★—☆



Mixed Meter ☆—☆

TONAL CONSTELLATION

Major Tonic ★—★—★—★



Major Dominant ☆—☆—★



Major Subdominant ☆

Pattern Display Format

RHYTHM PATTERNS LEARNED

Macro/Microbeat Patterns

- Duple Division: 4/12 mastered
- Triple Division: 0/8 mastered
- Unusual Pairs: 0/6 mastered

Function Patterns

- Elongation: 2/10 mastered
- Rest: 0/8 mastered
- Tie: 0/6 mastered

TONAL PATTERNS LEARNED

Major Tonality

- Tonic Function: 3/15 mastered
- Dominant Function: 0/12 mastered
- Subdominant: 0/10 mastered

VOCABULARY MASTERED

Dynamics: 2/8 terms

Articulation: 1/6 terms

Tempo: 0/10 terms

INTERVALS READY FOR MAGIC

- ✓ Unison
- ✓ Major 3rd
- × Minor 3rd (in progress)
- × Perfect 4th (locked)
- × Tritone (locked)
- × Perfect 5th (locked)
- × Minor 6th (locked)
- × Major 6th (locked)
- × Minor 7th (locked)
- × Major 7th (locked)

9. Assessment Integration

Regular Battles

- **Accuracy Required:** 85%

- **Pattern Count:** 3-5 patterns
- **Time Pressure:** Moderate
- **Failure:** Enemy escapes, can retry

Boss Battles

- **Accuracy Required:** 90%
- **Pattern Count:** 10-15 patterns
- **Time Pressure:** High
- **Multi-Phase:** Tests different skill types
- **Failure:** Boss retreats, player must practice more

Combat Flow

1. Enemy appears with weakness indicator (e.g., "Weak to: Major 3rd - Joy Magic")
2. Player selects appropriate interval
3. System records:
 - Accuracy (correct/incorrect)
 - Response time
 - What they chose if wrong (for commonErrors tracking)
4. Spell power based on performance
5. Update pattern confidence scores

Battle Integration Example

Phase 1: Enemy Appears

- Enemy shows weakness
- Adaptive System checks if player has encountered interval
- If new: Adds scaffolding (highlights on keyboard)
- If struggling: Con Man Wizard gives "accidental" hint
- If mastered: No hints, faster phase timer

Phase 2: Player Casts Spell

- Player selects interval on instrument display
- System records all performance data

Phase 3: Results

- Spell effect plays (damage/heal based on accuracy)
- Pattern progress updates in save file

10. Teacher Dashboard

Core Features (Client-Side Only)

- **No Data Storage:** Runs entirely in browser
- **Input:** Students upload save files to Google Classroom
- **Processing:** Teacher downloads zip, dashboard analyzes locally
- **Alternative:** Use Construct Master Collection plugins to interface with Google Drive API
- **Privacy:** No server-side data collection, all processing local

Analytics Available

1. Class Overview

- Average confidence by skill area
- Common struggling patterns
- Mastery distribution graphs
- Class-wide statistics

2. Individual Student View

- Pattern-by-pattern progress
- Time spent practicing
- Achievement history
- Specific error patterns
- Example: "Jamie: struggling with elongation patterns"

3. Custom Reports

- Filter by date range
- Group by skill type
- Export as PDF
- Email templates for parents
- Cross-class comparisons

Implementation Logic (Construct 3 Format)

Function: AnalyzeClassData

For each student save file in folder:

- Add to StudentArray

For each pattern in PatternList:

- Set TempStruggleCount to 0
- Set TempMasteryCount to 0

For each student in StudentArray:

- If student.patterns[pattern].confidence < 0.70
 - Add 1 to TempStruggleCount
- If student.patterns[pattern].confidence > 0.85
 - Add 1 to TempMasteryCount

- If TempStruggleCount > (StudentArray.width * 0.3)

- Add pattern to ClassStrugglePatterns

- Set ClassStats[pattern].AverageConfidence to (sum of all confidences / student count)

- Set ClassStats[pattern].MasteryRate to (TempMasteryCount / StudentArray.width)

Generate visual report using ClassStats data

11. Monetization Strategy

Core Principle

The game itself is always free. Revenue supports development and creator. "PBS Model" - public good supported by community.

Phase 1: Community Support (Immediate)

Platform: Patreon - "Support music education for all"

Tiers:

1. Chromatic Supporter (\$2/month)

- Name on supporters page on game website
- Access to private Discord channel
- Time commitment: Near-zero

2. Chromatic Insider (\$5/month)

- All previous rewards

- Monthly "Developer's Log" post (brief, informal update)
- Vote in minor content polls
- Time commitment: ~1 hour per month

3. **Chromatic Champion** (\$15/month)

- All previous rewards
- Name in game's official credits
- Access to early beta builds for testing
- Time commitment: Low (sharing builds already creating)

Phase 2: Teacher's Toolkit (After Kickstarter)

Concept: Premium subscription-based web dashboard for educators

Core Features:

1. Classroom Management: Create rosters, manage students
2. Progress Dashboard: Detailed analytics, Maestro's Reports
3. Assignment Tool: Assign specific arcs/mini-games as homework
4. Aggregate Insights: Class-wide data for lesson planning

Kickstarter Prerequisites:

- Successful Phase 1 with live game
- Active Patreon community as proof of concept
- Not asking for money to finish game (already free)
- Asking for support to build powerful, optional tools for teachers

Phase 3: Value-Added Content (Long-term)

Optional Downloads:

- Printable Lesson Plans & Worksheets (high-quality PDFs)
- Group Classroom Activities (non-digital games)
- Official Soundtrack (sheet music and audio files)
- Bundled into higher Teacher's Toolkit tier

12. Technical Specifications

Development Platform

- **Engine:** Construct 3
- **Audio Input:** Spacebar/controller/MIDI keyboard (NO audio recognition)
- **File Size:** ~100-150KB per save file (100 patterns = ~100KB)
- **Performance:** 60 FPS target on Chromebook
- **Browser:** Any modern browser (Chrome, Firefox, Safari, Edge)

Data Management

- **Save System:** Local with Google Drive sync option
- **Encryption:** Binary format with JSON conversion
- **Privacy:** No server-side data collection
- **File Format:** Encrypted binary (prevents easy tampering)

Event Sheet Organization

Main Event Sheets:

- Core_Game_Logic (gameplay mechanics)
- Adaptive_System_Functions (all learning functions)
- UI_Management (screens and feedback)
- Save_Load_System (data handling)
- Pattern_Library (content database)

Construct 3 Best Practices

1. **Use Dictionary objects for:**
 - Pattern data (key = patternId)
 - Vocabulary tracking
 - Interval accuracy
2. **Use Array objects for:**
 - Session history (2D array)
 - Review schedule
 - Pattern queues
3. **Use JSON object for:**
 - Loading/saving complete profiles
 - Parsing imported data
4. **Create reusable functions:**

- CalculateConfidence(accuracy, speed, streak)
- GetNextPatterns(category, count)
- UpdateMaestroReport()

5. Use local variables in functions:

- Keep calculations clean
- Avoid global variable conflicts

6. Add debug logging:

- Log all adaptive events
- Track decision points
- Help with testing

13. Future Features

Planned Additions

- **Two-player Competitive Modes:**
 - "Rhythm Duels" - split keyboard battles
 - "Interval Race" - who can identify faster
 - "Pattern Memory" - Simon Says with musical patterns
 - Clever keybinds on each side of Chromebook
 - No save file integration (separate games)
- **Additional Story Content:**
 - Episodes 1-7: Major Tonality
 - Episode 8+: Minor Introduction ("journey to minor kingdom")
 - Years 3-4: Mixolydian and Dorian modes
 - Years 5+: Other modes
 - Each adds new categories to save file dynamically
- **Community Features:**
 - Pattern sharing (no cost to implement)
 - Musical selfies (recorded performances)
 - Weekly challenges using existing patterns

Marketing Approach

- "Sesame Street meets Final Fantasy"

- "First 100% Free Music Curriculum Aligned Game"
- "Your Child's Music Teacher Helper"
- "Learn Music Like Learning a Language - Through Stories"
- "Empowering Music Teachers with Data-Driven Practice"
- "Where Gordon's MLT Meets Modern Gaming"

Outreach Strategy

1. Start with own classroom testing
2. Document everything with data
3. Show principal actual engagement results
4. Work with VAPA friend on district policies
5. Contact organizations:
 - NAFME (National Association for Music Education)
 - GIML (Gordon Institute for Music Learning)
 - State MEA (Music Educators Association)

14. Success Metrics & Research Foundation

Learning Outcomes (Based on Research)

- 85% pattern mastery rate (optimal accuracy threshold)
- 50-80% improved efficiency vs traditional methods
- 81.58% of students taking initiative to practice independently

Engagement Metrics

- Daily active users
- Session length (target: 15-20 minutes optimal)
- Story completion rate
- Pattern review compliance
- Spaced repetition adherence (2-7 day intervals)

Teacher Adoption

- Classroom implementations
- Teacher dashboard usage frequency

- Student progress reports generated
- Parent communication sent

Key Research Findings Implemented

- **85% Accuracy Rule:** Learning fastest at 15% error rate
- **Spaced Repetition:** 2-day minimum, then 4, 7, 14, 30 days
- **Session Design:** 10-15 minutes optimal, 3-5 patterns max
- **70/30 Rule:** 70% review, 30% new content
- **Zone of Proximal Development:**
 - Below 35% accuracy = reduce difficulty
 - 36-69% accuracy = maintain (in ZPD)
 - Above 70% accuracy = increase difficulty
- **Mastery Criteria:** 3-5 consecutive successful performances
- **Cognitive Load:** 3-4 chunks of information maximum
- **Motor Skills:** Interleaved practice within sessions
- **Lyrics vs Melody:** Spaced repetition works better for lyrics ($d=0.85$) than melody

Gordon's MLT Implementation

- Pattern progression: Unknown → Aural (Bum/Bah) → Verbal (syllables)
- Rhythm syllables: Du, Du-de, Du-ta-de-ta (NOT ti-ti ta)
- Tonal syllables: Do, Re, Mi, Fa, So, La, Ti
- Whole-part-whole approach in episode design
- Sequential learning with proper scaffolding
- Syllables as "magical spell ingredients"

Platform Performance

- Yousician: 20+ million monthly active users
 - SmartMusic: Instant color-coded feedback
 - SingMaster: 770,000+ sight-singing records analyzed
 - BKT effectiveness: 0.6865 AUC, 0.7472 accuracy
 - LSTM networks: 91.9% accuracy in performance evaluation
 - GIKT: 0.8480 AUC (highest performance)
-

Important Legal Notes

- Can reference "Based on Gordon's Music Learning Theory"
- Cannot quote Gordon's materials directly
- Must develop own implementations of MLT principles

This document serves as the complete reference for The Chromatic Lands development. All systems are designed to work together seamlessly, creating an educational experience where learning truly IS the gameplay.